



UNIVERSIDAD DE LOS LAGOS

LABORATORIO N°1: MIDIENDO LAS FALACIAS

INGENIERÍA CIVIL EN INFORMÁTICA
SISTEMAS DISTRIBUIDOS

Diego Leiva
Alan Gimilio

Docente Guía: Carlos Jesus Rojas Latorre

28 de agosto de 2026

Índice general

0.0.1. Entorno y método	1
0.0.2. Resultados	1
0.0.3. Observado vs esperado	2
0.0.4. La falacia que asumimos	2
0.0.5. Relación con la investigación	3

Índice de tablas

1.	Latencia inyectada vs. tiempo total de 100 llamadas	1
2.	Pérdida de paquetes inyectada vs. throughput y comportamiento del cliente . . .	1

0.0.1. Entorno y método

El laboratorio se ejecutó en un equipo con Windows, usando Docker Desktop (con backend WSL2/Linux) para levantar dos contenedores (`cliente` y `servidor`) conectados por la red `sd_net`, siguiendo el material provisto por el curso (`docker-compose.yml`, `cliente.py`, `servidor.py`).

Durante la puesta en marcha, Docker Desktop reportó el error `Virtualization support not detected`. La virtualización por hardware ya estaba habilitada en el equipo; lo que faltaba eran las características de Windows necesarias para que Docker pudiera usarla (`VirtualMachinePlatform` y el subsistema WSL). Se solucionó ejecutando PowerShell como administrador y corriendo:

```
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /
all /norestart
bcdedit /set hypervisorlaunchtype auto
```

seguido de un reinicio del equipo, tras lo cual Docker Desktop instaló el soporte de WSL faltante e inició correctamente.

La medición se realizó en cuatro pasos: (1) línea base con `ping` e `iperf3` sobre la red sin modificar (2) 100 y 1000 llamadas PING/PONG secuenciales con la red sana (3) inyección de latencia con `tc qdisc netem delay` en 50, 200 y 500 ms, midiendo el total de 100 llamadas en cada nivel (4) inyección de pérdida de paquetes con `tc qdisc netem loss` en 1 %, 5 %, 20 % y 100 %, midiendo throughput con `iperf3` y comportamiento del cliente en cada nivel, incluyendo la falla provocada con `loss 100 %`.

0.0.2. Resultados

Latencia <code>tc</code>	Total (s)	Promedio (ms)	Máx (ms)
0 ms (base)	0.011	0.1	1.4
50 ms	5.041	50.4	51.3
200 ms	20.045	200.4	200.9
500 ms	50.046	500.5	501.0

Tabla 1: Latencia inyectada vs. tiempo total de 100 llamadas

Pérdida <code>tc</code>	Throughput <code>iperf3</code>	Total cliente (s)
1 %	10.7 Gbit/s	0.011
5 %	449 Mbit/s	1.459
20 %	1.21 Mbit/s	5.252
100 %	–	error inmediato / timeout

Tabla 2: Pérdida de paquetes inyectada vs. throughput y comportamiento del cliente

0.0.3. Observado vs esperado

En el Paso 3 (latencia), los cálculos teóricos ($N \times \text{RTT}$ para 100 llamadas secuenciales) coincidieron casi exactamente con lo medido en los tres niveles: 50 ms $\rightarrow$ 5.0 s esperado vs 5.041 s medido; 200 ms $\rightarrow$ 20 s esperado vs 20.045 s medido; 500 ms $\rightarrow$ 50 s esperado vs 50.046 s medido. La degradación fue perfectamente lineal, sin sorpresas: el cliente espera cada respuesta antes de enviar la siguiente llamada, por lo que el retardo se acumula de forma directa.

En el Paso 4 (pérdida) las diferencias fueron mayores a lo esperado intuitivamente. Con solo 1% de pérdida, el throughput de `iperf3` ya cayó a menos de la mitad (de 23.2 Gbit/s a 10.7 Gbit/s), lo que no es proporcional al porcentaje de pérdida: se explica porque TCP reduce agresivamente su ventana de congestión ante la primera señal de pérdida. En cambio, el cliente de 100 llamadas no mostró cambios visibles con 1% de pérdida (total casi igual al base), porque con pocos paquetes chicos es probable que ninguno se perdiera en esa corrida particular. Esto muestra que la cantidad de tráfico determina si una falla de baja probabilidad estadística llega a manifestarse.

0.0.4. La falacia que asumimos

La falacia identificada es “**La red es confiable**” (Deutsch/Gosling, falacia #1). Se ve claramente en `cliente.py`, en la línea

```
TIMEOUT = float(os.getenv("TIMEOUT_S", "0")) or None
```

donde, al no definirse `TIMEOUT_S`, la expresión `0 or None` se evalúa como `None`, dejando el socket **sin timeout** en la llamada `socket.create_connection`. Esto se traduce directamente en la línea donde el cliente espera cada respuesta:

```
respuesta = f.readline() # aquí el cliente ESPERA a la red
```

`readline()` es una llamada bloqueante: si el servidor no responde, el programa se queda esperando indefinidamente, sin ningún mecanismo propio para detectar que la red falló. Esto se comprobó en la falla provocada (`loss 100%`): sin `TIMEOUT_S`, el programa no mostró ningún error de aplicación propio dependió de un error externo del sistema (`Errno 113 No route to host`) con `TIMEOUT_S=3`, el cliente abortó de forma controlada a los 3.0 s, mostrando que el mecanismo de timeout sí existe en el código, pero **no está activado por defecto**.

El cambio que corregiría esta falacia es simple: cambiar el valor por defecto de `TIMEOUT_S` de `"0"` a un valor explícito razonable (por ejemplo, 5 segundos), de modo que la ausencia de configuración no implique esperar para siempre. Además, se podría agregar lógica de reintento con backoff exponencial y un número máximo de intentos antes de reportar la falla al usuario, en vez de terminar con un único error tras el timeout.

0.0.5. Relación con la investigación

La aplicación investigada para la Evaluación 1 es **Transbank / Webpay**, uno de los pagos electrónicos más usada en Chile. En su flujo de integración (Webpay Plus), el comercio debe realizar una llamada de *confirmación* (`commit`) al servidor de Transbank tras la autorización del pago para saber si la transacción fue aprobada.

La falacia que más ha afectado a este tipo de sistema es **“la red es confiable”**, la misma identificada en el Paso 4 de este laboratorio. Si la llamada de confirmación falla por timeout o pérdida de paquetes, el comercio no puede asumir que la transacción no ocurrió: el cargo puede haberse efectuado igualmente del lado del banco emisor, y solo se perdió la respuesta de vuelta, el mismo fenómeno de retransmisión silenciosa que se observó con `tc netem loss` en este laboratorio, donde el cliente completaba llamadas con éxito pero con máximos disparados sin que la aplicación se enterara del retraso real. Por eso la documentación de integración de Transbank recomienda no asumir un resultado negativo ante un timeout, sino volver a consultar el estado real de la transacción mediante un endpoint de `status` antes de decidir si reintentar el cobro. Un manejo ingenuo de este timeout (asumiendo directamente que “no llegó respuesta = no se cobró”) puede llevar a cobros duplicados o a discrepancias entre lo que el comercio y el banco registran como transacción exitosa.